assign-day-7

July 11, 2023

1 Assignment Problems: Day 7

The following problems require access to the file named 'Dataset_Day7.csv', provided with this assignment. (Use Random State = 203)

The datasets include data from 768 women with several medical predictor variables and one target variable('Outcome'). The classification goal is to predict whether or not the patients in the dataset have diabetes or not.

For columns: Glucose, BloodPressure, BMI, DiabetesPedigreeFunction

If the column value is 0, then they should be considered as missing data.

[1]:
```
import numpy as np
import pandas as pd
from math import log
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, log_loss, precision_recall_curve,␣
 ↪accuracy_score, precision_score, recall_score
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
```

- Firstly, replace all Missing values with relevant figures.
- Then remove all existing outliers and get the final data for classification.

[2]:
```
data = pd.read_csv("Dataset_Day7.csv")
data.skew()
```

[2]:
```
Pregnancies                 0.901674
Glucose                     0.173754
BloodPressure              -1.843608
BMI                        -0.428982
DiabetesPedigreeFunction    1.919911
Age                         1.129597
Outcome                     0.635017
dtype: float64
```

[3]:
```
data = pd.read_csv("Dataset_Day7.csv")

# Missing data treatment
print('Before Missing Data Treatment')
for col in ['Glucose', 'BloodPressure', 'BMI', 'DiabetesPedigreeFunction'] :
    print(col,' = ',len(data[col][data[col]==0]))
for col in ['Glucose', 'BloodPressure', 'BMI', 'DiabetesPedigreeFunction'] :
    data[col][data[col]==0] = data[col].median()
print('\nAfter Missing Data Treatment')
for col in ['Glucose', 'BloodPressure', 'BMI', 'DiabetesPedigreeFunction'] :
    print(col,' = ',len(data[col][data[col]==0]))

# Standard scaling of the continuous columns only
for i in ['BMI', 'DiabetesPedigreeFunction', 'Glucose', 'Pregnancies',␣
 ↪'BloodPressure', 'Age'] :
    data[i] = (data[i] - data[i].mean())/data[i].std()
outlier = data[(data["BMI"]>3)|(data["BMI"]<-3)|
               ␣
 ↪(data["Pregnancies"]>3)|(data["Pregnancies"]<-3)|(data["Age"]>3)|(data["Age"]<-3)|
               ␣
 ↪(data["Glucose"]>3)|(data["Glucose"]<-3)|(data["BloodPressure"]>3)|(data["BloodPressure"]<-
#              ␣
 ↪(data["DiabetesPedigreeFunction"]>3)|(data["DiabetesPedigreeFunction"]<-3)]
OutlierFreeData = data.drop(outlier.index,axis=0)
print("\nno. of Outlier",len(outlier), "\n% of Outlier = ", str(len(outlier)/
 ↪len(data)*100))
```

```
Before Missing Data Treatment
Glucose  =  5
BloodPressure  =  35
BMI  =  11
DiabetesPedigreeFunction  =  0

After Missing Data Treatment
Glucose  =  0
BloodPressure  =  0
BMI  =  0
DiabetesPedigreeFunction  =  0

no. of Outlier 20
% of Outlier =  2.604166666666667
```

```
C:\Users\nikhi\AppData\Local\Temp\ipykernel_8964\1164204054.py:8:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
```

```
data[col][data[col]==0] = data[col].median()
```

- Split the data into 70% training and 30% testing data. Then, create a logistic regression model with target variable as 'Outcome'.
- Print the default model performance metrics: Accuracy, Precision, Recall, F1Score & AIC

[4]:
```
OutlierFreeData.corr()
```

[4]:
```
                          Pregnancies   Glucose  BloodPressure       BMI  \
Pregnancies                  1.000000  0.113921       0.214220  0.025710
Glucose                      0.113921  1.000000       0.204934  0.229207
BloodPressure                0.214220  0.204934       1.000000  0.300720
BMI                          0.025710  0.229207       0.300720  1.000000
DiabetesPedigreeFunction    -0.022521  0.124618      -0.009852  0.130085
Age                          0.554971  0.266316       0.342124  0.053570
Outcome                      0.209358  0.493898       0.171304  0.296052

                          DiabetesPedigreeFunction       Age   Outcome
Pregnancies                             -0.022521  0.554971  0.209358
Glucose                                  0.124618  0.266316  0.493898
BloodPressure                           -0.009852  0.342124  0.171304
BMI                                      0.130085  0.053570  0.296052
DiabetesPedigreeFunction                 1.000000  0.042410  0.174249
Age                                      0.042410  1.000000  0.252315
Outcome                                  0.174249  0.252315  1.000000
```

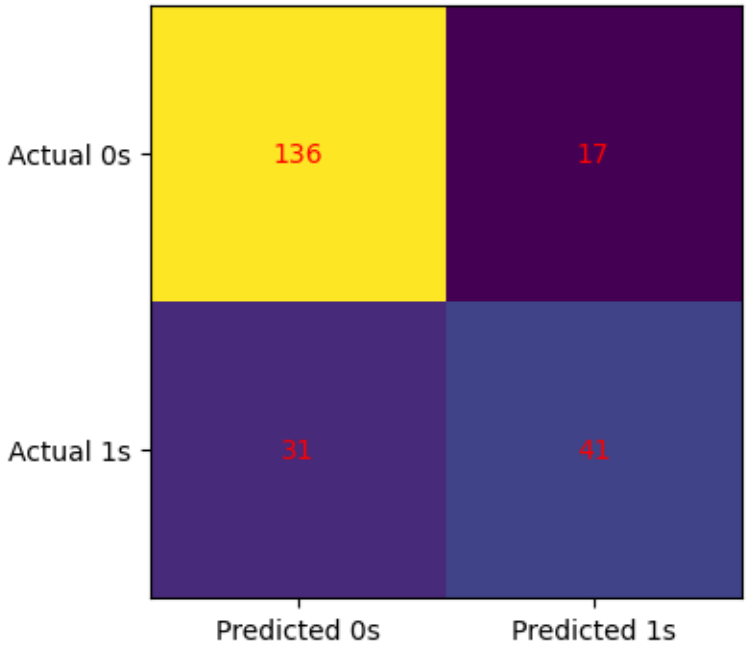
[5]:
```
# Spliting of data 7:3
X = OutlierFreeData.drop("Outcome",axis=1)
y = OutlierFreeData["Outcome"]
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.30,␣
 ↪random_state=42)

# Logistic Regression model
model = LogisticRegression(random_state=45)
model.fit(X_train, y_train)
y_pred = model.predict(X_test) #default threshold value is 0.5
f1_score = 2*precision_score(y_test,y_pred)*recall_score(y_test,y_pred)/
 ↪(precision_score(y_test,y_pred)+recall_score(y_test,y_pred))
print("Accuracy = ", accuracy_score(y_test,y_pred)*100)
print("Precision = ", precision_score(y_test,y_pred)*100)
print("Recall = ", recall_score(y_test,y_pred)*100)
print("F1-Score = ", f1_score*100)
print("AIC = ", (2/len(y_pred)*log_loss(y_test,y_pred)/len(y_pred) +␣
 ↪2*len(X_train.columns)/len(y_pred))*100)

# confusion matrix plotting
cm = confusion_matrix(y_test, y_pred)
fig, ax = plt.subplots(figsize=(4, 4))
```

```
ax.imshow(cm)
#ax.grid(False)
ax.xaxis.set(ticks=(0, 1), ticklabels=('Predicted 0s', 'Predicted 1s'))
ax.yaxis.set(ticks=(0, 1), ticklabels=('Actual 0s', 'Actual 1s'))
ax.set_ylim(1.5, -0.5)
for i in range(2):
    for j in range(2):
        ax.text(j, i, cm[i, j], ha='center', va='center', color='red')
plt.show()
```

```
Accuracy =  78.66666666666666
Precision =  70.6896551724138
Recall =  56.94444444444444
F1-Score =  63.07692307692307
AIC =  5.363710865078565
```



- Plot a F1_score vs threshold curve. Find the threshold for which f1-score is the highest.

[6]:
```
y_scores = model.predict_proba(X_test)[:,1]
prec, rec, tre = precision_recall_curve(y_test,y_scores)
f_score = (2*prec*rec)/(prec+rec)
index = np.where(f_score==max(f_score))
y_pred = (y_scores >= tre[index]).astype(int)

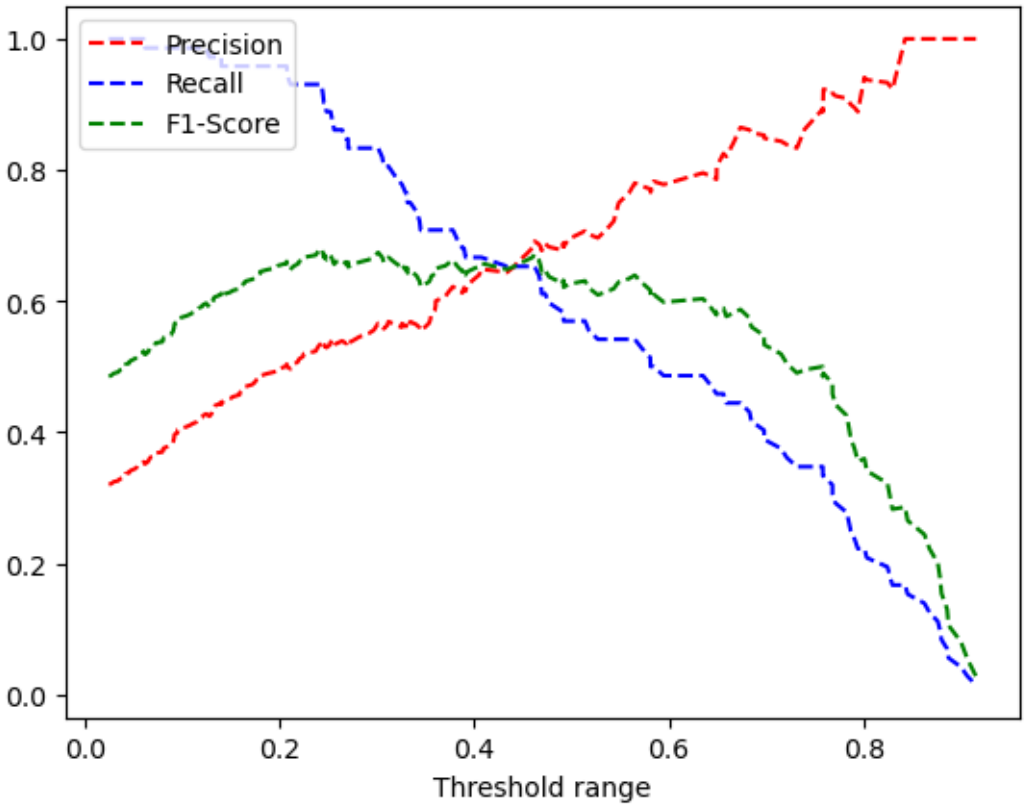
# Plotting graph
plt.plot(tre,prec[:-1],'r--',label='Precision')
```

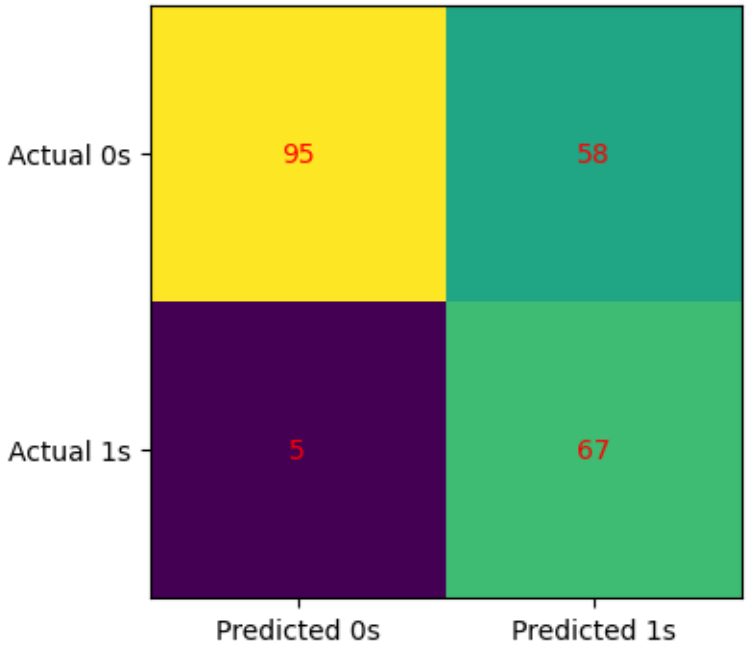
```
plt.plot(tre,rec[:-1],'b--',label='Recall')
plt.plot(tre,f_score[:-1],'g--',label='F1-Score')
plt.xlabel('Threshold range')
plt.legend(loc='upper left')


print("For Optimum Threshold = ", tre[index][0])
print("Accuracy = ", accuracy_score(y_test,y_pred)*100)
print("Precision = ", prec[index][0]*100)
print("Recall = ", rec[index][0]*100)
print("F1-Score = ", max(f_score)*100)
print("AIC = ", (2/len(y_pred)*log_loss(y_test,y_pred)/len(y_pred) +␣
 ↪2*len(X_train.columns)/len(y_pred))*100)

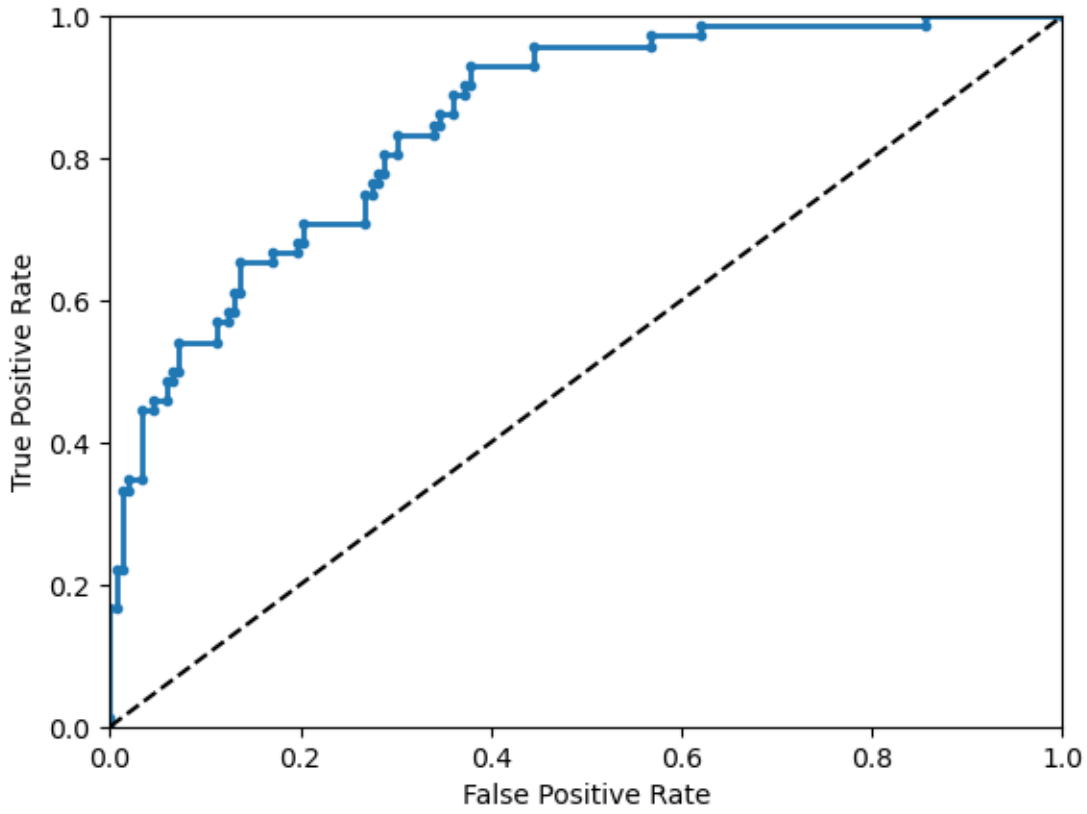
# confusion matrix plotting
cm = confusion_matrix(y_test, y_pred)
fig, ax = plt.subplots(figsize=(4, 4))
ax.imshow(cm)
#ax.grid(False)
ax.xaxis.set(ticks=(0, 1), ticklabels=('Predicted 0s', 'Predicted 1s'))
ax.yaxis.set(ticks=(0, 1), ticklabels=('Actual 0s', 'Actual 1s'))
ax.set_ylim(1.5, -0.5)
for i in range(2):
    for j in range(2):
        ax.text(j, i, cm[i, j], ha='center', va='center', color='red')
plt.show()
```

```
For Optimum Threshold =  0.24291375709875984
Accuracy =  72.0
Precision =  53.6
Recall =  93.05555555555556
F1-Score =  68.02030456852792
AIC =  5.37320384374895
```


Precision
Recall
F1-Score
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
Threshold range


Actual 0s
Actual 1s
95
58
5
67
Predicted 0s
Predicted 1s

[7]:
```
y_scores = model.predict_proba(X_test)
fpr, tpr, threshold= roc_curve(y_test, y_scores[:,1])
def plot_roc_curve(fpr, tpr):
    plt.clf()
    plt.plot(fpr, tpr, linewidth=2, marker='.', label='Logistic')
    plt.plot([0,1], [0,1], 'k--')
    plt.axis([0,1,0,1])
    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
plot_roc_curve(fpr, tpr)
plt.show()
```



Name - Nikhil Khatri

SIC - 21BCSA92